

УЧЕБЕН МАТЕРИАЛ

Урок № 1: Въведение в алчните алгоритми. Основни принципи и примери

Предмет: Алгоритми и структури от данни

1. Въведение и цели на темата

В практическата разработка на софтуер оптимизационните задачи са ежедневие — от разпределяне на сървърен ресурс и кеширане до изчисляване на ресто и маршрутизация. Алчните алгоритми (Greedy Algorithms) са бърз, интуитивен и мощен инструмент, който взема най-изгодното локално решение в текущия момент с надеждата да постигне глобален оптимум.

След този урок ще можете да:

- Дефинирате същността на алчния алгоритмичен подход и ключовите понятия „локално оптимален избор“ и „оптимална подструктура“;
- Разграничавате алчния подход от метода на пълното изчерпване (brute force) и да разпознавате границите на неговата приложимост;
- Имплементирате, проследявате и тествате работещо конзолно решение на Python за класическата задача за рестото;
- Прилагате добри практики при работа с колекции, целочислено деление ($//$) и остатък ($\%$).

2. Теоретичен конспект

2.1. Основни понятия и дефиниции

Понятие	Описание
Алчен алгоритъм (Greedy Algorithm)	Подход за решаване на задачи, при който на всяка стъпка се избира непосредствено най-изгодната алтернатива без връщане назад.
Локално оптимален избор (Greedy Choice Property)	Изборът, който изглежда най-добър в конкретната стъпка (напр. вземане на най-големия възможен номинал монета).
Оптимална подструктура (Optimal Substructure)	Свойство на задачата, при което оптималното крайно решение се състои от оптималните решения на нейните по-малки подзадачи.
Пълно изчерпване (Brute Force)	Подход, който генерира и проверява абсолютно всички възможни комбинации, за да намери абсолютния оптимум (значително

Понятие	Описание
	по-бавен).

2.2. Принцип на работа: Сравнение на алгоритмичните стратегии

[Задача: Разпредели остатъка S]

Повтори.

- **АЛЧЕН ПОДХОД:**
Вземи най-големия възможен елемент веднага → Извади от S →
(Бърз: $O(N)$, праволинеен, работи за канонични системи)
- **ПЪЛНО ИЗЧЕРПВАНЕ:**
Генерирай всяка възможна комбинация → Преброй елементите →
Избери най-малката.
(Бавен: експоненциална сложност $O(2^N)$, но гарантира оптимум при всякакви системи)

Типични стъпки при алчен алгоритъм:

- 1. Подреждане (Sorting):** Кандидатите се сортират по дефинирания критерий за изгода (най-често низходящо).
- 2. Итерация:** Последователно преглеждане на елементите.
- 3. Проверка за допустимост:** Може ли текущият елемент да бъде използван спрямо оставащия лимит?
- 4. Акумулиране и редуциране:** Вземане на елемента и намаляване на търсения остатък.
- 5. Край:** Процесът спира, когато остатъкът стане равен на 0 или кандидатите се изчерпят.

2.3. Демонстрационен пример: Задача за рестото на Python

Дадено е ресто за връщане $S = 87$ \$ лв. Наличните номинали в паричната система са: \$50, 20, 10, 5, 2, 1\$. Търси се минималният брой банкноти и монети.

```
# =====
# ДЕМО ПРОГРАМА: Изчисляване на ресто чрез алчен алгоритъм
# =====

def calculate_change(amount: int, denominations: list[int]) -> dict[int, int]:
    """
    Изчислява минималния брой номинали за връщане на ресто чрез алчен подход.

    :param amount: Сума за връщане (цяло неотрицателно число)
    :param denominations: Списък с налични номинали
    :return: Речник с броя на използваните номинали {номинал: брой}
    """
    # 1. Гарантираме подредба от най-големия към най-малкия номинал
    sorted_coins = sorted(denominations, reverse=True)

    change_distribution = {}
```

```

remaining = amount

for coin in sorted_coins:
    # 2. Локално оптимален избор: колко пъти се съдържа най-големият
    възможен номинал
    count = remaining // coin

    if count > 0:
        change_distribution[coin] = count
        # 3. Редуциране на задачата до по-малък остатък
        remaining = remaining % coin

# 4. Проверка дали цялата сума е била успешно разпределена
if remaining > 0:
    print(f"Внимание: остават неразпределени {remaining} лв. поради липсващ
номинал!")

return change_distribution

# --- ТЕСТВАНЕ НА РЕШЕНИЕТО ---
target_amount = 87
standard_coins = [50, 20, 10, 5, 2, 1]

result = calculate_change(target_amount, standard_coins)

print(f"Разпределение на рестото за {target_amount} лв.:")
total_count = 0
for coin, count in result.items():
    print(f" - {count} бр. x {coin} лв.")
    total_count += count

print(f"Общ брой номинали: {total_count}")

```

Поетапно проследяване (Trace Table) за 87 лв.:

Съпка	Разглеждан номинал	Изчисление (remaining // coin)	Взети броя	Нов остатък (remaining % coin)
1	50	$\$87 // 50 = 1\$$	1	$\$87 \% 50 = 37\$$
2	20	$\$37 // 20 = 1\$$	1	$\$37 \% 20 = 17\$$
3	10	$\$17 // 10 = 1\$$	1	$\$17 \% 10 = 7\$$
4	5	$\$7 // 5 = 1\$$	1	$\$7 \% 5 = 2\$$
5	2	$\$2 // 2 = 1\$$	1	$\$2 \% 2 = 0\$$
6	1	$\$0 // 1 = 0\$$	0	$\$0\$$ (край)

2.4. Внимание: Чести грешки и добри практики

□ Капанът на алчния подход (Кога НЕ работи?):

Алчният избор **не гарантира** оптимално решение за произволни системи от номинали!

Контрапример: Представете си парична система с номинали: [4, 3, 1] и ресто за връщане 6.

- **Алчен избор:** първо взема 4, остават 2 $\rightarrow$ взема 1 + 1 $\rightarrow$ **3 монети** (4, 1, 1).
- **Глобален оптимум:** 3 + 3 $\rightarrow$ **2 монети!**
Извод: При нестандартни системи се прилага **Динамично оптимизиране**, а не алчен алгоритъм.

Добри практики при програмиране:

1. **Не мутирайте входните данни:** Използвайте `sorted(coins, reverse=True)` вместо `coins.sort()`, за да запазите първоначалния масив непокътнат.
2. **Използвайте целочислени оператори:** Избягвайте изваждане в цикъл `while`, когато можете да пресметнете броя директно с `//` и `%` за $O(1)$ време на номинал.
3. **Валидирайте граничните случаи:** Тествайте алгоритъма със сума 0, сума по-малка от най-малкия номинал и с непокътнати входни списъци.

3. Cheatsheet / Речник на новите термини

Термин / Синтаксис	Описание / Значение	Пример в Python
<code>//</code>	Целочислено деление (връща цялата част без остатък)	<code>87 // 50</code> $\rightarrow$ 1
<code>%</code>	Деление с остатък (модулен оператор)	<code>87 % 50</code> $\rightarrow$ 37
<code>sorted(iterable, reverse=True)</code>	Вградена функция, връщаща нов списък в низходящ ред	<code>sorted([10, 50, 5], reverse=True)</code> $\rightarrow$ [50, 10, 5]
<code>dict.items()</code>	Метод за итериране по двойки ключ-стойност	<code>for coin, count in result.items():</code>
Локален оптимум	Най-добрият избор за текущата итерация	Избор на максималната монета $\leq$ остатък
Глобален оптимум	Най-доброто крайно решение за цялата задача	Минимален брой монети за цялата сума

4. Въпроси и задачи за самопроверка

Контролни въпроси:

1. Каква е съществената разлика между алчен алгоритъм и търсене с пълно изчерпване (brute force)?

Отговор:

2. Кой е локално оптималният избор при разпределяне на ресто от 43 лв. при номинали [50, 20, 10, 5, 2, 1]?

Отговор:

3. Защо номиналите трябва да бъдат разгледани строго в низходящ ред?

Отговор:

4. Вярно ли е твърдението: „Алчният алгоритъм винаги намира абсолютното най-добро решение за всяка възможна задача“? Обосновете се.

Отговор:

Практически предизвикателства:

Задача 1 (Базово ниво):

Напишете функция `get_minimal_coins(amount)`, която връща само **общия брой монети** (едно цяло число), необходими за връщане на сума `amount` при стандартни номинали [50, 20, 10, 5, 2, 1].

Тестов пример: `get_minimal_coins(93)` трябва да върне 5 (тъй като $93 = 50 + 20 + 20 + 2 + 1$).

Задача 2 (Ниво с предизвикателство):

Представете си банкомат, който зарежда касети с банкноти от 100, 50, 20 и 10 лв. Ако клиент заяви сума 135 лв., как ще реагира програмата при алчен избор? Добавете проверка в кода, която връща съобщение за грешка: „Сумата не може да бъде изплатена с наличните номинали!“, ако след края на цикъла `remaining > 0`.